

TP2_SAE4D01_SFLOW_MALOT_VALENTIN

TP2 : Télémétrie sflow

Synthèse :

Durant ce TP, j'ai découvert le protocole sflow et je l'ai mis en oeuvre dans deux environnements différents : Une machine virtuelle et un routeur conteneurisé.

Dans la première partie, j'ai installé `hsflowd` qui collecte les métriques et envoie des datagrammes UDP vers un ou plusieurs collecteurs (à modifier dans le fichier de configuration). Le collecteur que j'ai utilisé s'appelle `sflow-rd` et donne accès à un browser pour parcourir les métriques et à une API REST également. J'ai également utilisé NuShell afin de mettre en forme les données en temps réel.

Durant la dernière partie, j'ai utilisé le protocole sFlow avec des images de routeurs pour déployer une petite architecture leaf and spine. Chaque image embarquait un agent sFlow automatiquement configuré pour envoyer les données vers le collecteur.

L'ensemble des deux TP m'a donc permis de mieux comprendre deux approches complémentaires du monitoring réseau : sFlow pour l'analyse du trafic et gNMI pour la supervision et la configuration des équipements.

Travail Pratique :

1. Qu'est-ce que sflow ?

sFlow (sampled Flow) est un protocole de télémétrie réseau qui permet de surveiller le trafic sur des équipements réseau (switches, routeurs). Il fonctionne

par échantillonnage statistique : au lieu de capturer chaque paquet, il en prélève un sur N (ex. 1 paquet sur 1000), ce qui le rend très léger en termes de ressources. Les données collectées (en-têtes de paquets, compteurs d'interfaces) sont envoyées à un collecteur sFlow pour analyse. Il est défini dans la RFC 3176.

2. Quelles sont les différences entre les protocoles sFlow, NetFlow et IPFIX ?

Critère	sFlow	NetFlow	IPFIX
Origine	InMon Corp / RFC 3176	Cisco (propriétaire)	IETF (RFC 5101) — standard
Méthode	Échantillonnage de paquets	Agrégation de flux	Agrégation de flux
Granularité	Paquet (header + payload)	Flux (5-tuple)	Flux (extensible)
Précision	Statistique (approximatif)	Exacte sur les flux	Exacte sur les flux
Charge CPU	Très faible	Modérée	Modérée
Extensibilité	Limitée	Limitée (Cisco)	Très extensible (templates)
Matériels	Multi-vendor	Principalement Cisco	Multi-vendor
Latence	Très faible	Plus élevée	Plus élevée

En résumé : sFlow est plus léger mais moins précis ; IPFIX est le standard ouvert qui succède à NetFlow v9.

3. Est-ce que sFlow est open source ?

sFlow est un protocole (spécifié par une RFC publique), mais pas un logiciel open source en lui-même. C'est la RFC 3176 qui traite de ce protocole. En revanche, les implémentations varient : certaines sont open source (ex.

sflowtool , sflow-rt en partie), d'autres sont propriétaires (agents intégrés dans les ASICs des switches)

Le site **sflow.org** maintenu par InMon Corp publie les spécifications et des outils de référence.

4. Quel est le port et le type de protocole utilisé par un collecteur sflow ?

- **Port : UDP 6343** (port officiel IANA)
- **Protocole transport : UDP**

Les agents sFlow sur les équipements envoient des datagrammes UDP vers l'adresse IP du collecteur sur ce port. Le format des datagrammes est défini par la spécification sFlow v5.

5. Sur quels types de matériels peut-on utiliser sflow ?

sFlow est implémenté directement dans le hardware de nombreux équipements :

- **Switches** : Arista, Juniper, HP/Aruba, Dell, Brocade, Extreme Networks, Mellanox
- **Routeurs** : Juniper MX, certains Cisco (via adaptations)
- **Hyperviseurs** : Open vSwitch (OVS) supporte nativement sFlow
- **Systèmes Linux** : via le module noyau `hsflowd` (Host sFlow)
- **Équipements WiFi** : certains points d'accès Aruba/HP

L'avantage majeur de sFlow est que l'échantillonnage est réalisé **dans l'ASIC** et non par le CPU, ce qui le rend utilisable même sur des liens à très haut débit (100G+) sans impact sur les performances de forwarding.

2. Utilisation de sflow pour surveiller une machine virtuelle Linux Debian

1. En suivant <https://sflow.net/downloads.php> installez `hsflowd`

J'ai récupéré le lien de téléchargement pour le paquet Debian 13 sur le site <https://sflow.net/downloads.php>.

-> https://github.com/sflow/host-sflow/releases/download/v2.1.17-1/hsflowd-debian12_2.1.17-1_amd64.deb

```
wget https://github.com/sflow/host-sflow/releases/download/v2.1.17-1/hsflowd-debian12_2.1.17-1_amd64.deb
dpkg -i hsflowd-debian12_2.1.17-1_amd64.deb
sudo systemctl enable hsflowd
sudo systemctl start hsflowd
```

2. En suivant <https://sflow-rt.com/download.php#download> et <https://sflow-rt.com/intro>. php installez sflow-rt.

D'après la documentation :

```
sudo apt update
sudo apt install default-jre
wget https://inmon.com/products/sFlow-RT/sflow-rt_3.2-1729.deb
sudo dpkg -i sflow-rt_3.2-1729.deb
sudo systemctl enable sflow-rt
sudo systemctl start sflow-rt
```

L'interface de sFlow-RT est accessible à l'IP suivante :

<http://10.202.8.2:8008/html/index.html#status>

3. Configurez le fichier /etc/hsflowd.conf afin d'alimenter sflow-rt. Aidez-vous de <https://sflow.net/host-sflow-linux-config.php>. Vous pouvez "débugger" la configuration de hsflowd avec la commande `hsflowd -dddd`.

```
cat > /etc/hsflowd.conf << 'EOF'
sflow {
    polling = 30
    sampling = 400
    collector { ip = 127.0.0.1 udpport = 6343 }
```

```
pcap { dev = eth0 }  
}  
EOF
```

J'ai vérifié que l'agent soit bien détecté par `sflow-rt : curl`
`http://localhost:8008/agents/json :`

```
root@debian13vm:/usr/local/sflow-rt# curl  
http://localhost:8008/agents/json  
{  
  "10.202.8.2": {  
    "sFlowDatagramsLost": 0,  
    "sFlowDatagramSource": [  
      "127.0.0.1"  
    ],  
    "sFlowNotificationSamples": 0,  
    "sFlowFlowDuplicateSamples": 0,  
    "sFlowDatagramsReceived": 6,  
    "sFlowNotificationDrops": 0,  
    "sFlowFlowSamples": 6,  
    "sFlowDatagramsOutOfOrder": 0,  
    "sFlowNotificationDuplicateSamples": 0,  
    "sFlowFlowDrops": 0,  
    "sFlowFlowLostSamples": 0,  
    "sFlowNotificationLostSamples": 0,  
    "sFlowCounterSamples": 0,  
    "sFlowFlowDatasources": 1,  
    "sFlowCounterOutOfOrderSamples": 0,  
    "sFlowNotificationOutOfOrderSamples": 0,  
    "sFlowDatagramsReset": 0,  
    "firstSeen": 19897,  
    "sFlowCounterDatasources": 1,  
    "sFlowFlowOutOfOrderSamples": 0,  
    "sFlowNotificationDatasources": 0,  
    "uptime": 17467,  
    "sFlowCounterDuplicateSamples": 0,  
    "lastSeen": 7571,  
    "sFlowDatagramsDuplicates": 0,
```

```
"sFlowCounterLostSamples": 0
}
}
```

4. Utilisez le flow-browser de sflow-rt afin de visualiser les flux de votre machine virtuelle (port de destination , adresses IP source et destination...).

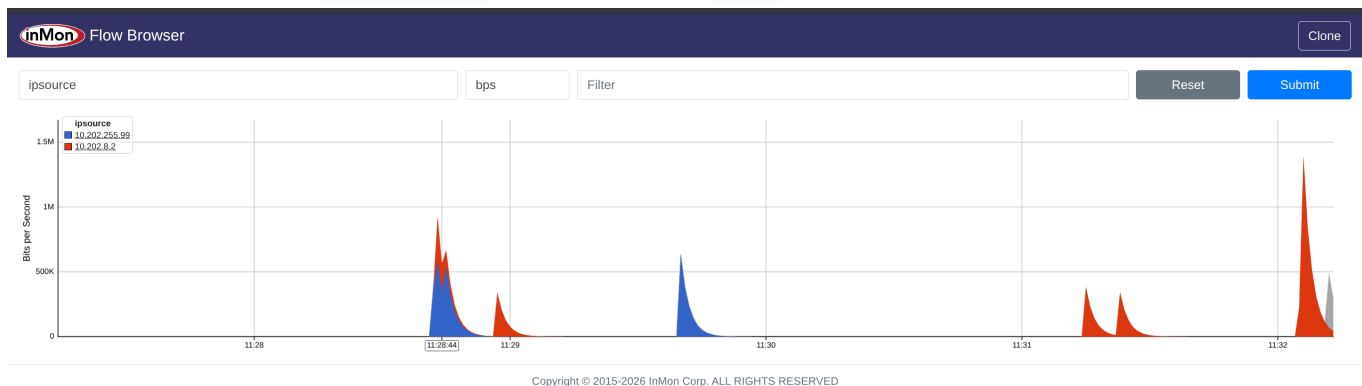
J'ai installé le module flow-browser de sflow-rt car il était inaccessible par défaut.

```
cd /usr/local/sflow-rt
sudo ./get-app.sh sflow-rt flow-browser
sudo systemctl restart sflow-rt
```

Je me suis rendu à l'URL suivant :

```
http://10.202.8.2:8008/app/flow-browser/html/index.html
```

J'ai indiqué la Clé `ipsource` et en Value `bps` .



5. Retrouvez à l'aide de l'API Browser de sflow-rt les métriques produites par sflow. Récupérez la liste des métriques via curl

```
# Repérer les agents connus
curl http://localhost:8008/agents/json
# Liste des métriques disponibles
curl http://localhost:8008/metrics/json
# Métriques d'un agent spécifique (par exemple 127.0.0.1)
curl http://localhost:8008/metrics/127.0.0.1/json
```

6. Installez sampleflowtrend avec Docker. Utilisez-le pour alimenter votre compte-rendu par des analyses intéressantes

J'ai installé sampleflowtrend avec Docker, cependant, j'ai modifié le fichier de configuration `/etc/hsflowd.conf` comme ceci :

```
root@debian13vm:/usr/local/sflow-rt# cat /etc/hsflowd.conf
sflow {
    polling = 30
    sampling = 400
    collector { ip = 127.0.0.1 udpport = 6343 }
    pcap { dev = eth0 }
    collector {
        ip = 127.0.0.1
        udpport = 6344
    }
}
```

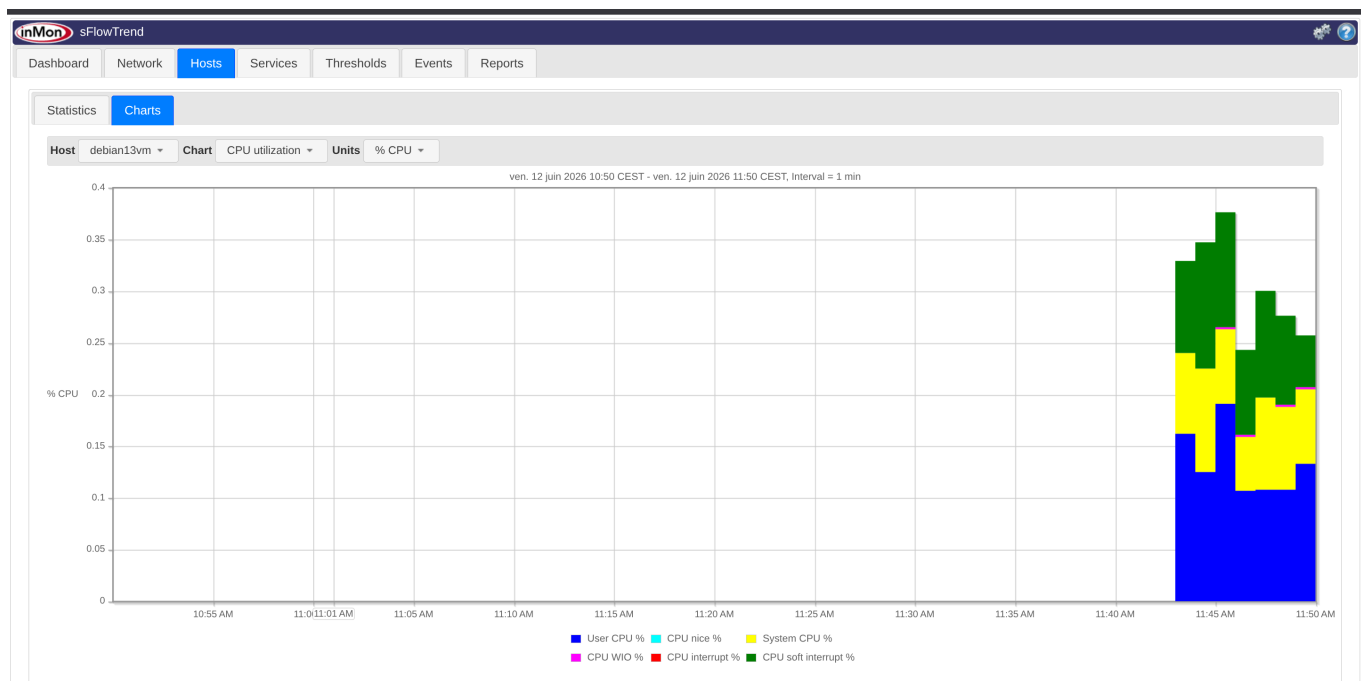
```
systemctl restart hsflowd
```

Puis j'ai lancé la commande fournie dans le TP :

```
docker run -v /var/local/sflowtrend-pro:/var/local/sflowtrend-pro \
-p 6344:6343/udp \
-p 8087:8087 \
-p 8443:8443 \
-h sflowtrend-pro \
-e TZ=Europe/Paris \
-d --restart unless-stopped \
sflow/sflowtrend
```

Et je me suis rendu sur la page suivante

<http://10.202.8.2:8087/sflowtrend/> > Hosts > Charts > debian13vm



Sur la capture d'écran ci-dessus, j'observe un diagramme type battons qui évolue au cours du temps avec les différentes charges CPU : User, System, Interruptions... Ce graphique est mis à jour toutes les minutes.

7. Installez sflowtool (voir <https://github.com/sflow/sflowtool>).

```
# Depuis les sources (GitHub)
apt install -y gcc make autoconf libtool

git clone https://github.com/sflow/sflowtool.git
cd sflowtool
./boot.sh
./configure
make
make install

# Vérifier
sflowtool -h
```

8. Utilisez-le pour réceptionner les paquets sflow de votre machine physique sur le port 6344 afin d'éviter la collision de port avec "sflow-rt".

Le port 6344 était déjà utilisé par le conteneur Docker, j'ai donc utilisé le port 6345 :


```
sflowtool -p 6345
```

J'ai aussi modifié le fichier de configuration du collecteur :

```
cat > /etc/hsflowd.conf << 'EOF'
sflow {
    polling = 30
    sampling = 400
    collector { ip = 127.0.0.1 udpport = 6343 }
    collector { ip = 127.0.0.1 udpport = 6344 }
    collector { ip = 127.0.0.1 udpport = 6345 }
    pcap { dev = eth0 }
    tcp {}
    docker { markTraffic = on }
}
EOF

systemctl restart hsflowd
systemctl status hsflowd
```

Je constate que je reçois bien les datagrammes dans mon terminal :

```
startDatagram =====
datagramSourceIP 127.0.0.1
datagramSize 792
unixSecondsUTC 1781258251
unixSecondsUTC_uS 689117
localtime 2026-06-12T11:57:31+0200
datagramVersion 5
agentSubId 100000
agent 10.202.8.2
packetSequenceNo 3
sysUpTime 19704
samplesInPacket 1
startSample -----
sampleType_tag 0:4
sampleType COUNTERSSAMPLE
```

sampleSequenceNo 1
sourceId 2:1
counterBlock_tag 0:2100
vnode_mhz 3302
vnode_cpus 6
vnode_memory 8332857344
vnode_memory_free 2089136128
vnode_num_domains 1
counterBlock_tag 0:2001
adaptor_0_ifIndex 4
adaptor_0_MACs 1
adaptor_0_MAC_0 e2331e2e8f18
adaptor_1_ifIndex 2
adaptor_1_MACs 1
adaptor_1_MAC_0 080027594bb5
adaptor_2_ifIndex 3
adaptor_2_MACs 1
adaptor_2_MAC_0 c66ceba34183
counterBlock_tag 0:2010
udpInDatagrams 828
udpNoPorts 31
udpInErrors 0
udpOutDatagrams 921
udpRcvbufErrors 0
udpSndbufErrors 0
udpInCsumErrors 0
counterBlock_tag 0:2009
tcpRtoAlgorithm 1
tcpRtoMin 200
tcpRtoMax 120000
tcpMaxConn 4294967295
tcpActiveOpens 377
tcpPassiveOpens 64
tcpAttemptFails 12
tcpEstabResets 16
tcpCurrEstab 3
tcpInSegs 610987
tcpOutSegs 591094
tcpRetransSegs 767

tcpInErrs 0
tcpOutRsts 307
tcpInCsumErrors 0
counterBlock_tag 0:2008
icmpInMsgs 275
icmpInErrors 51
icmpInDestUnreachs 0
icmpInTimeExcds 16
icmpInParamProbs 0
icmpInSrcQuenchs 0
icmpInRedirects 0
icmpInEchos 0
icmpInEchoReps 0
icmpInTimestamps 208
icmpInAddrMasks 0
icmpInAddrMaskReps 0
icmpOutMsgs 0
icmpOutErrors 0
icmpOutDestUnreachs 247
icmpOutTimeExcds 0
icmpOutParamProbs 0
icmpOutSrcQuenchs 0
icmpOutRedirects 37
icmpOutEchos 0
icmpOutEchoReps 0
icmpOutTimestamps 0
icmpOutTimestampReps 0
icmpOutAddrMasks 210
icmpOutAddrMaskReps 0
counterBlock_tag 0:2007
ipForwarding 1
ipDefaultTTL 64
ipInReceives 627556
ipInHdrErrors 0
ipInAddrErrors 89
ipForwDatagrams 1517
ipInUnknownProtos 0
ipInDiscards 0
ipInDelivers 625931

ipOutRequests 557662
ipOutDiscards 0
ipOutNoRoutes 40
ipReasmTimeout 0
ipReasmReqds 10
ipReasmOKs 5
ipReasmFails 0
ipFragOKs 0
ipFragFails 0
ipFragCreates 0
counterBlock_tag 0:2005
disk_total 32626910208
disk_free 14774948864
disk_partition_max_used 85.07
disk_reads 133381
disk_bytes_read 4083822592
disk_read_time 37436
disk_writes 289489
disk_bytes_written 23750496256
disk_write_time 99313
counterBlock_tag 0:2004
mem_total 8332857344
mem_free 2089136128
mem_shared 0
mem_buffers 290906112
mem_cached 4634185728
swap_total 8587833344
swap_free 8587833344
page_in 1995245
page_out 11596922
swap_in 0
swap_out 0
counterBlock_tag 0:2003
cpu_load_one 0.060
cpu_load_five 0.060
cpu_load_fifteen 0.100
cpu_proc_run 0
cpu_proc_total 413
cpu_num 6

```
cpu_speed 3302
cpu_uptime 13581
cpu_user 663200
cpu_nice 100
cpu_system 383880
cpu_idle 79925680
cpu_wio 27200
cpuintr 0
cpu_sintr 184370
cpuinterrupts 16784551
cpu_contexts 14784127
cpu_steal 0
cpu_guest 0
cpu_guest_nice 0
counterBlock_tag 0:2006
nio_bytes_in 29387
nio_pkts_in 290
nio_errs_in 0
nio_drops_in 0
nio_bytes_out 26551
nio_pkts_out 194
nio_errs_out 0
nio_drops_out 0
counterBlock_tag 0:2000
hostname debian13vm
UUID 162d1b84-a0e7-e242-b5cb-151709f55a6c
machine_type 3
os_name 2
os_release 6.12.88+deb13-amd64
endSample -----
endDatagram =====
```

9. Utilisez NuShell pour afficher les flux sflow remontés en temps réel au format json avec la commande suivante :

#	counterBlock_tag	udpInDatagrams	udpNoPorts	udpInErrors	udpOutDatagrams	udpRcvbufErrors	udpSndbufErrors	udpInCsumErrors	ifSpeed
16	0:2010	1043	65	0	1250	0	0	0	False

#	counterBlock_tag	ifIndex	networkType	ifSpeed	ifDirection	ifStatus	ifInOctets	ifInUcastPkts	ifInMulticastPkts	ifInBroadcastPkts	ifInDiscards	...
17	0:1	2	6	1000000000	1	3	996800	9582	4294967295	4294967295	0	...

#	counterBlock_tag	udpInDatagrams	udpNoPorts	udpInErrors	udpOutDatagrams	udpRcvbufErrors	udpSndbufErrors	udpInCsumErrors	ifSpeed
18	0:2010	1055	65	0	1266	0	0	0	False

#	counterBlock_tag	ifIndex	networkType	ifSpeed	ifDirection	ifStatus	ifInOctets	ifInUcastPkts	ifInMulticastPkts	ifInBroadcastPkts	ifInDiscards	...
19	0:1	2	6	1000000000	1	3	1020647	9820	4294967295	4294967295	0	...

#	counterBlock_tag	udpInDatagrams	udpNoPorts	udpInErrors	udpOutDatagrams	udpRcvbufErrors	udpSndbufErrors	udpInCsumErrors	ifSpeed
20	0:2010	1064	65	0	1278	0	0	0	False

#	counterBlock_tag	ifIndex	networkType	ifSpeed	ifDirection	ifStatus	ifInOctets	ifInUcastPkts	ifInMulticastPkts	ifInBroadcastPkts	ifInDiscards	...
21	0:1	2	6	1000000000	1	3	1047727	10074	4294967295	4294967295	0	...

#	counterBlock_tag	udpInDatagrams	udpNoPorts	udpInErrors	udpOutDatagrams	udpRcvbufErrors	udpSndbufErrors	udpInCsumErrors	ifSpeed
22	0:2010	1073	65	0	1290	0	0	0	False

#	counterBlock_tag	ifIndex	networkType	ifSpeed	ifDirection	ifStatus	ifInOctets	ifInUcastPkts	ifInMulticastPkts	ifInBroadcastPkts	ifInDiscards	...
---	------------------	---------	-------------	---------	-------------	----------	------------	---------------	-------------------	-------------------	--------------	-----

```

nu
sflowtool -p 6345 -j
  | from json --objects
  | flatten samples --all
  | select -i elements
  | flatten --all
  | default False udpInDatagrams
  | default False ifSpeed
  | filter { $in.udpInDatagrams != False or $in.ifSpeed !=
False }

```

10. Utilisez la commande tee de NuShell pour sauvegarder le flux des résultats dans un fichier csv.

Voici la commande que j'ai utilisé :

```

sflowtool -p 6345 -j
  | from json --objects
  | flatten samples --all
  | select -i elements
  | flatten --all
  | default False udpInDatagrams
  | default False ifSpeed

```

```
| filter { $in.udpInDatagrams != False or $in.ifSpeed !=  
False }  
| tee { to csv | save -f resultats_sflow.csv }
```

Le résultat obtenu est bien celui attendu :

```
~> ls
```

#	name	type	size	modified
0	TP_GNMIC	dir	4,0 kB	4 hours ago
1	TP_SFLOW	dir	4,0 kB	2 hours ago
2	resultats_sflow.csv	file	553 B	now
3	test	dir	4,0 kB	4 hours ago

```
~> cat resultats_sflow.csv
```

```
counterBlock_tag,ifIndex,networkType,ifSpeed,ifDirection,ifStatu  
s,ifInOctets,ifInUcastPkts,ifInMulticastPkts,ifInBroadcastPkts,i  
fInDiscards,ifInErrors,ifInUnknownProtos,ifOutOctets,ifOutUcastP  
kts,ifOutMulticastPkts,ifOutBroadcastPkts,ifOutDiscards,ifOutErr  
ors,ifPromiscuousMode,udpInDatagrams,udpNoPorts,udpInErrors,udpO  
utDatagrams,udpRcvbufErrors,udpSndbufErrors,udpInCsumErrors  
0:1,2,6,1000000000,1,3,805710,4170,4294967295,4294967295,0,0,429  
4967295,792633,3042,4294967295,4294967295,0,0,0,False,,,,,  
0:2010,,,False,,,,,,,,,,,,,89,29,0,144,0,0,0
```

3. Utilisation de sflow pour la métrologie de routeurs Arista

1. Installez containerlab avec curl sur votre machine virtuelle (voir <https://containerlab.dev/install/>)

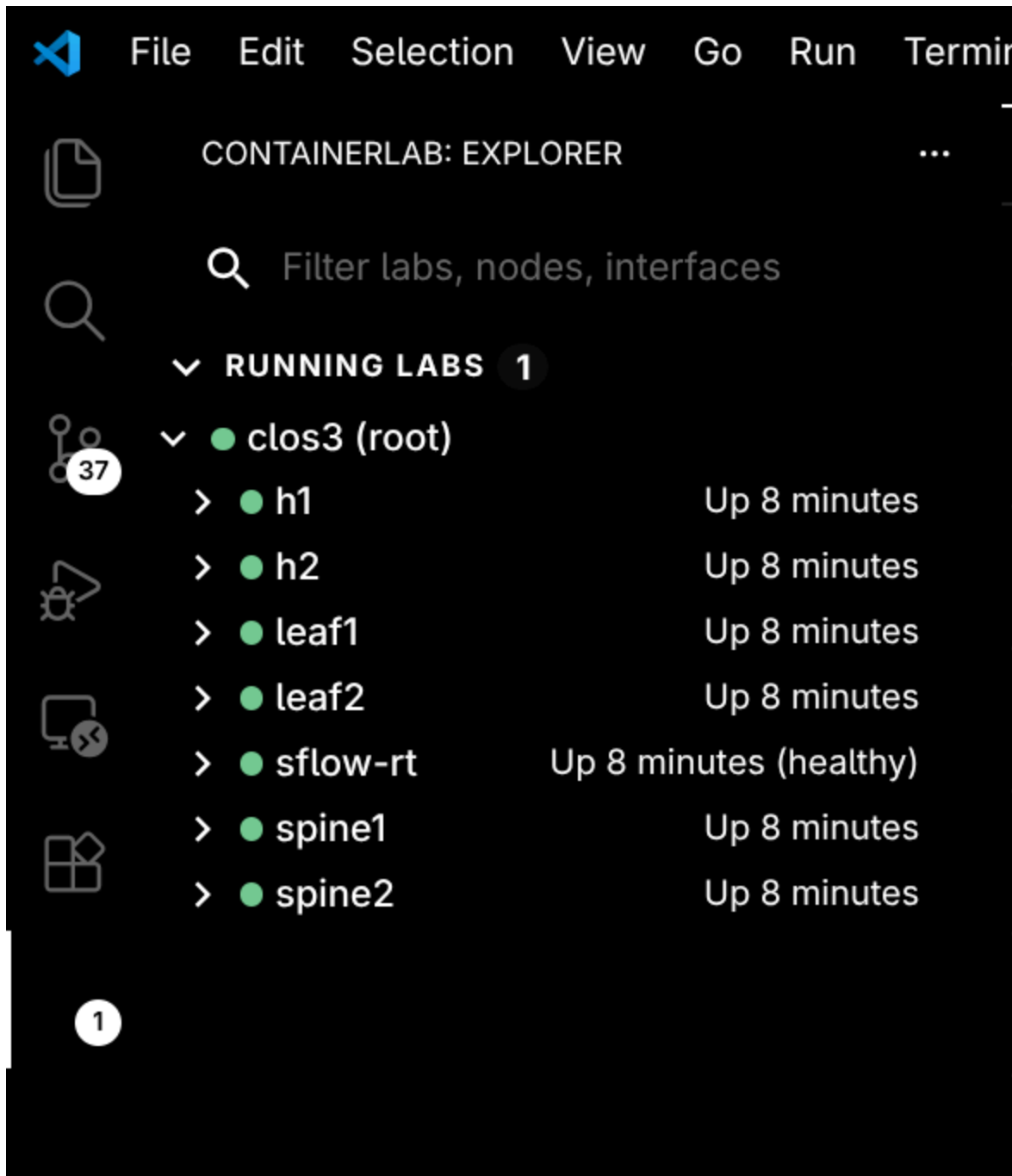
Voici la commande (Containerlab étant déjà installé sur ma VM)

```
curl -sL https://containerlab.dev/setup | sudo -E bash -s "all"
```

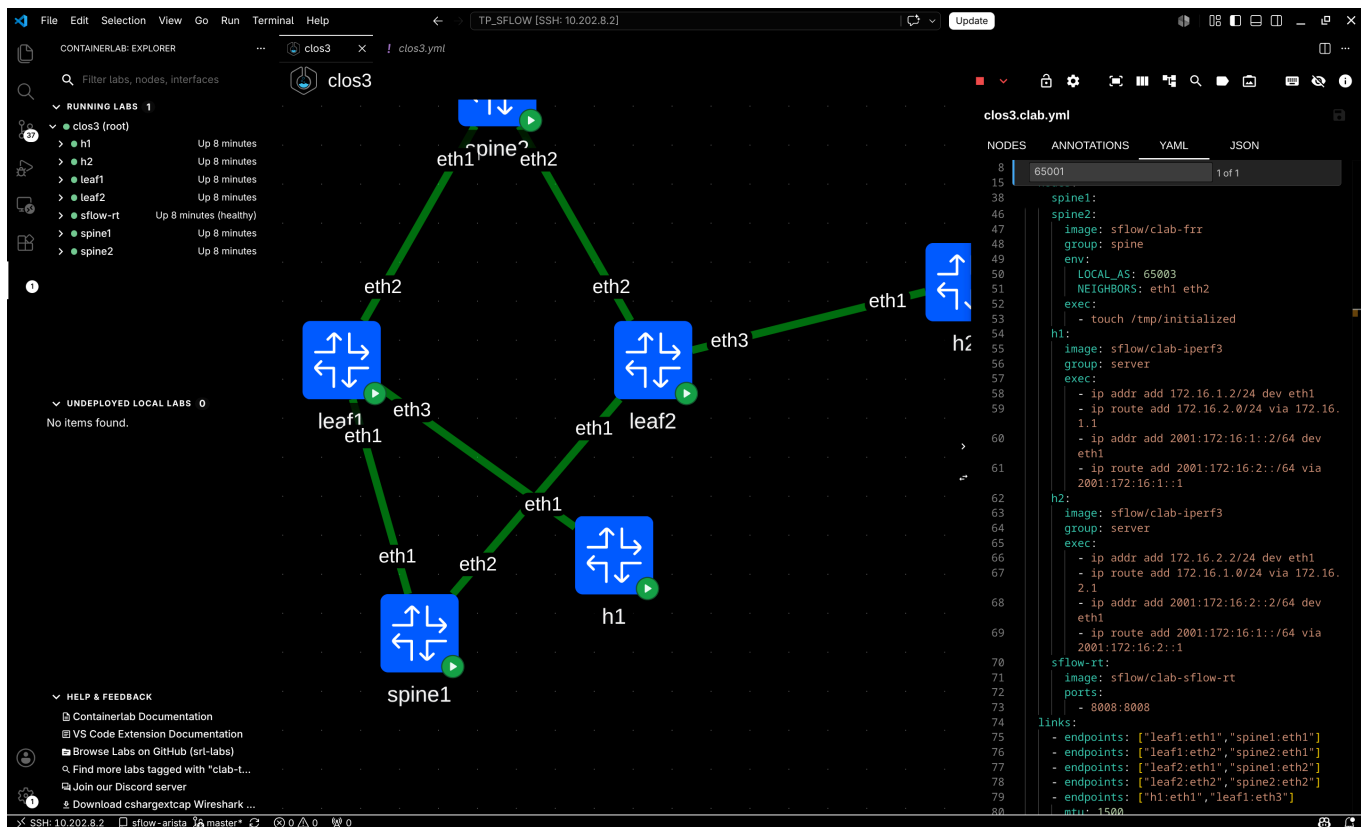
2. Clonez le dépôt <https://github.com/pushou/sflow-arista.git>

```
git clone https://github.com/pushou/sflow-arista.git
```

3. Ouvrez le projet avec vscode. Installez l'extension "containerlab" pour vscode.



4. Utilisez cette extension pour visualiser le projet.

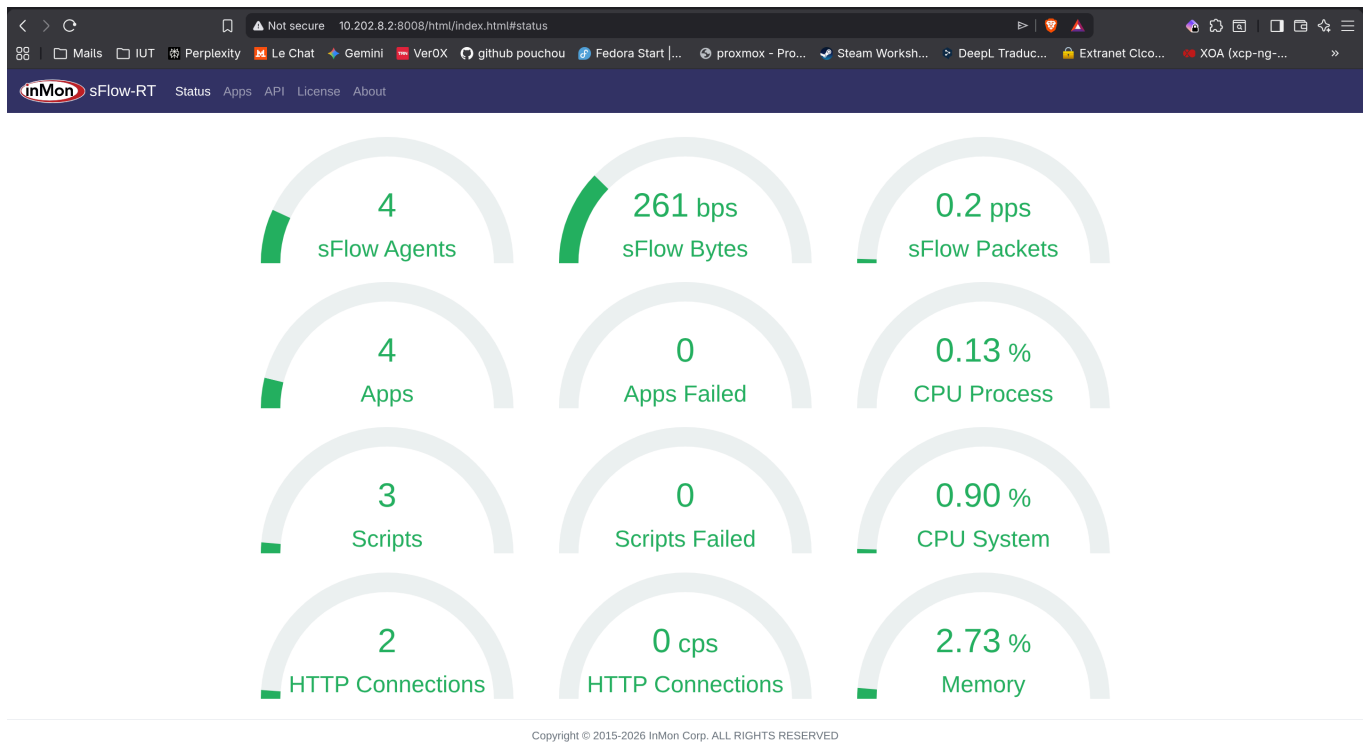


5. Chargez la topologie "3 Stage Clos Topology" et lancez-la.

OK (Voir screenshot ci-dessus)

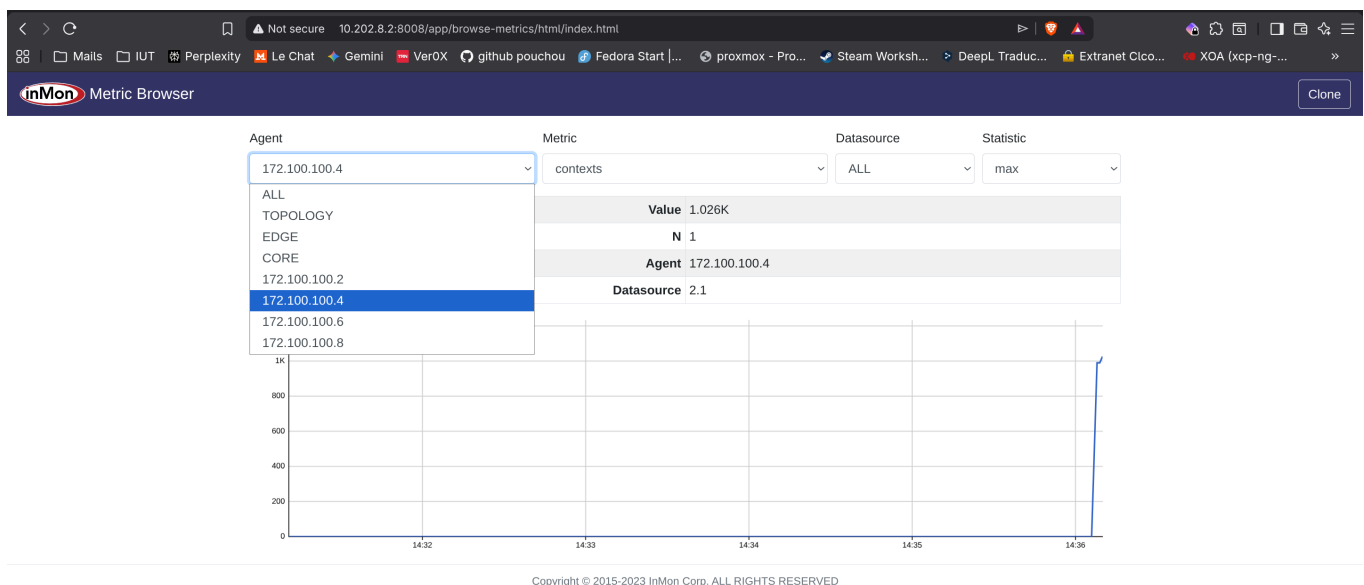
6. Retrouvez l'affichage web de sflow-rt afin de visualiser la métrologie des flux

J'ai retrouvé l'interface de `sflow-rt` déployée par containerlab :



Je constate sur l'image ci-dessus que j'ai quatre agents qui sont détectés par le connecteur.

Dans la rubrique Apps > Browse-metrics, je retrouve les 4 adresses IP des routeurs déployés via containerlab. En effet, chacun possède bien un daemon type sflow (par exemple hsflowd) qui expose les métriques à un collecteur (sflow-rt).



7. Voici le schéma descriptif du fonctionnement :

